

# B.C.A. (Sem. II) Examination, 2026

(Session : 2025-28)

2 BCA 2

## Advanced Programming in C

Paper Code : 211602

### COMPLETE MODEL QUESTION PAPER WITH STEP-BY-STEP SOLUTIONS

Time : Three Hours

Full Marks : 100

Pass Marks : 40

**Note :** Candidates are required to give their answers in their own words as far as practicable. The questions are of equal value. Answer any five questions.

#### Question 1

Differentiate structure and union with suitable example. Also explain Array in structure using program.

#### Solution / Explanation:

- A. **Difference between Structure and Union:** A **structure** allocates separate memory for each member, allowing all members to be accessed simultaneously. A **union** allocates a single shared memory location equal to the size of its largest member, allowing only one member to store a valid value at any given time.

| Feature           | Structure (struct)                                    | Union (union)                                       |
|-------------------|-------------------------------------------------------|-----------------------------------------------------|
| Memory Allocation | Allocates sum of sizes of all members (plus padding). | Allocates size of the single largest member.        |
| Member Storage    | Every member gets its own unique memory space.        | All members share the same memory location.         |
| Member Access     | All members can be accessed at any time.              | Only one member can be accessed reliably at a time. |
| Keyword           | <b>struct</b>                                         | <b>union</b>                                        |
| Initialization    | All members can be initialized together.              | Only the first member can be initialized.           |

```
1 #include <stdio.h>
2
3 struct DataStruct {
4     int i;           // 4 bytes
5     float f;         // 4 bytes
6     char str[20];    // 20 bytes
7 }; // Total size = 28 bytes
8
9 union DataUnion {
10     int i;           // 4 bytes
11     float f;         // 4 bytes
12     char str[20];    // 20 bytes
13 }; // Total size = 20 bytes (largest member)
14
15 int main() {
16     printf("Size of struct: %lu bytes\n", sizeof(struct DataStruct));
17     printf("Size of union: %lu bytes\n", sizeof(union DataUnion));
18     return 0;
```

```
19 }
```

Listing 1: Example comparing Struct and Union memory allocation

- B. Array in Structure (Array as a Structure Member):** An array can be embedded inside a structure definition to represent multiple data items of the same type under a single structure field (e.g., storing marks of 5 subjects for a student).

```
1 #include <stdio.h>
2
3 struct Student {
4     int roll_no;
5     char name[30];
6     int marks[5]; // Array as a structure member
7 };
8
9 int main() {
10     struct Student s1 = {101, "Rahul Sharma", {85, 90, 78, 88, 92}};
11     int total = 0;
12
13     printf("Student Roll No: %d\nName: %s\n", s1.roll_no, s1.name);
14     printf("Marks in 5 subjects: ");
15     for(int i = 0; i < 5; i++) {
16         printf("%d ", s1.marks[i]);
17         total += s1.marks[i];
18     }
19     printf("\nTotal Marks: %d / 500\n", total);
20     return 0;
21 }
```

Listing 2: C Program demonstrating Array inside a Structure

---

## Question 2

Write a C program using an array of structures to store details of 10 books such as Book-ID, Title, Author and Price. Display the details of books whose price is greater than a value entered by the user.

### Solution / Explanation:

An **array of structures** stores multiple structure variables sequentially in memory. In this program, **struct Book** defines the fields, and an array **struct Book books[10]** holds the records.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Book {
5     int book_id;
6     char title[50];
7     char author[50];
8     float price;
9 };
10
11 int main() {
12     struct Book books[10];
13     float threshold;
14
15     // Input details for 10 books
16     printf("=== Enter Details of 10 Books ===\n");
```

```

17     for (int i = 0; i < 10; i++) {
18         printf("\nBook %d:\n", i + 1);
19         printf("Enter Book ID: ");
20         scanf("%d", &books[i].book_id);
21         getchar(); // consume newline character
22
23         printf("Enter Title: ");
24         fgets(books[i].title, sizeof(books[i].title), stdin);
25         books[i].title[strcspn(books[i].title, "\n")] = 0; // remove trailing
newline
26
27         printf("Enter Author: ");
28         fgets(books[i].author, sizeof(books[i].author), stdin);
29         books[i].author[strcspn(books[i].author, "\n")] = 0;
30
31         printf("Enter Price: ");
32         scanf("%f", &books[i].price);
33     }
34
35     // Input user price threshold
36     printf("\n-----\n");
37     printf("Enter minimum price threshold to filter books: ");
38     scanf("%f", &threshold);
39
40     // Display matching books
41     printf("\n== Books with Price Greater Than %.2f ==\n", threshold);
42     int count = 0;
43     for (int i = 0; i < 10; i++) {
44         if (books[i].price > threshold) {
45             printf("\nBook ID : %d\n", books[i].book_id);
46             printf("Title   : %s\n", books[i].title);
47             printf("Author  : %s\n", books[i].author);
48             printf("Price   : $%.2f\n", books[i].price);
49             count++;
50         }
51     }
52
53     if (count == 0) {
54         printf("No books found exceeding price $%.2f\n", threshold);
55     }
56
57     return 0;
58 }

```

Listing 3: Complete C Program for Array of Structures (Book Details)

### Question 3

Explain dynamic memory allocation in C. Describe malloc (), calloc (), realloc () and free () with syntax and examples.

**Solution / Explanation:**

**Dynamic Memory Allocation (DMA)** refers to allocating memory manually at runtime from the **Heap** segment rather than compile time from the Stack. It allows programs to request exact memory sizes dynamically using functions declared in <stdlib.h>.

1. **malloc() (Memory Allocation):** Allocates a single contiguous block of specified bytes. Memory contains garbage values initially.

- **Syntax:** `void* malloc(size_t size);`
  - **Example:** `int *ptr = (int*) malloc(5 * sizeof(int));`
2. **calloc() (Contiguous Allocation):** Allocates multiple blocks of memory for an array and automatically initializes every bit to zero.
- **Syntax:** `void* calloc(size_t num_elements, size_t element_size);`
  - **Example:** `int *ptr = (int*) calloc(5, sizeof(int));`
3. **realloc() (Re-allocation):** Resizes an previously allocated memory block without losing existing data.
- **Syntax:** `void* realloc(void* ptr, size_t new_size);`
  - **Example:** `ptr = (int*) realloc(ptr, 10 * sizeof(int));`
4. **free() (Deallocation):** Releases allocated dynamic memory back to the heap, preventing **memory leaks**.
- **Syntax:** `void free(void* ptr);`
  - **Example:** `free(ptr); ptr = NULL;`

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int *arr;
6     int n = 3;
7
8     // 1. Allocate memory using malloc
9     arr = (int*) malloc(n * sizeof(int));
10    if (arr == NULL) { printf("Memory allocation failed!\n"); return 1; }
11
12    for(int i = 0; i < n; i++) arr[i] = (i + 1) * 10;
13
14    // 2. Resize memory using realloc from 3 to 5 elements
15    n = 5;
16    arr = (int*) realloc(arr, n * sizeof(int));
17    arr[3] = 40; arr[4] = 50;
18
19    printf("Dynamically Allocated Array: ");
20    for(int i = 0; i < n; i++) printf("%d ", arr[i]);
21    printf("\n");
22
23    // 3. Free memory
24    free(arr);
25    arr = NULL;
26    return 0;
27 }

```

Listing 4: Complete C Program demonstrating DMA functions

## Question 4

Explain any two of the following with suitable examples: (a) Enumerations and typedef (b) Nested structures (c) Self-referential structures (d) Chain pointers

### Solution / Explanation:

- (a) **Enumerations (enum) and typedef:** An enum is a user-defined type consisting of integral constants with symbolic names. typedef creates an alias name for existing data types to improve code readability.

```
1 typedef enum { LOW, MEDIUM, HIGH } Priority;
2 Priority current = HIGH; // current has value 2
```

Listing 5: Example of Enum and Typedef

- (b) **Nested Structures:** A structure embedded inside another structure to represent hierarchical data relationships.

```
1 struct Address { char city[20]; int pin; };
2 struct Employee {
3     int id;
4     char name[30];
5     struct Address addr; // Nested structure
6 };
```

Listing 6: Example of Nested Structure

- (c) **Self-Referential Structures:** A structure that contains a pointer member pointing to a structure of its own type. Used in dynamic data structures like Linked Lists and Trees.

```
1 struct Node {
2     int data;
3     struct Node *next; // Pointer to self-type structure
4 };
```

Listing 7: Example of Self-Referential Structure

- (d) **Chain Pointers (Pointer to Pointer):** A pointer variable that stores the memory address of another pointer variable (e.g., int \*\*pptr).

```
1 int x = 10;
2 int *p = &x; // Pointer to int
3 int **pp = &p; // Pointer to pointer to int
4 printf("Value of x = %d", **pp); // Prints 10
```

Listing 8: Example of Chain Pointers

---

## Question 5

**How Pointers are created and initialized? Explain Advantages of pointer and call by value and call by reference.**

### Solution / Explanation:

- A. **Creation and Initialization of Pointers:** A pointer is created by placing an asterisk (\*) before the variable name during declaration. It is initialized by assigning the memory address of a variable using the address-of operator (&).

```
1 int num = 25;
2 int *ptr; // Creation (Declaration)
3 ptr = &num; // Initialization (Stores address of num)
```

Listing 9: Pointer Creation and Initialization

- B. **Advantages of Pointers:**

- Enables dynamic memory allocation at runtime.
- Enables Call by Reference, allowing functions to modify actual arguments.
- Efficient handling of arrays, strings, and dynamic data structures (Linked Lists, Trees).
- Reduces memory overhead and speeds up execution when passing large structures to functions.

### C. Call by Value vs. Call by Reference:

- **Call by Value:** Copies the actual parameter's value into formal parameters. Changes inside the function do not affect actual caller variables.
- **Call by Reference:** Passes memory addresses of actual parameters as pointers. Changes made inside the function directly modify original variables.

```

1 #include <stdio.h>
2
3 void swapByValue(int a, int b) {
4     int temp = a; a = b; b = temp;
5 }
6
7 void swapByReference(int *a, int *b) {
8     int temp = *a; *a = *b; *b = temp;
9 }
10
11 int main() {
12     int x = 10, y = 20;
13     swapByValue(x, y);
14     printf("After Swap by Value: x = %d, y = %d\n", x, y); // x=10, y=20
15
16     swapByReference(&x, &y);
17     printf("After Swap by Reference: x = %d, y = %d\n", x, y); // x=20, y
18                             =10
19     return 0;
20 }

```

Listing 10: Call by Value vs Call by Reference (Swapping Example)

---

## Question 6

Explain recursion in C. Write recursive functions to generate the first  $n$  terms of the Fibonacci sequence.

### Solution / Explanation:

**Recursion** is a programming technique where a function calls itself directly or indirectly to break a problem into smaller sub-problems. Every recursive function must contain: 1. **Base Condition:** Stops recursion and prevents infinite stack overflow. 2. **Recursive Step:** Solves smaller instances of the original problem.

**Fibonacci Series Definition:**  $F(0) = 0$ ,  $F(1) = 1$ , and  $F(n) = F(n - 1) + F(n - 2)$  for  $n \geq 2$ .

```

1 #include <stdio.h>
2
3 // Recursive function returning the n-th Fibonacci number
4 int fibonacci(int n) {
5     if (n == 0) return 0; // Base case 1
6     if (n == 1) return 1; // Base case 2
7     return fibonacci(n - 1) + fibonacci(n - 2); // Recursive step
8 }

```

```

9
10 int main() {
11     int n = 8; // Number of terms
12     printf("First %d terms of Fibonacci sequence:\n", n);
13     for (int i = 0; i < n; i++) {
14         printf("%d ", fibonacci(i));
15     }
16     printf("\n");
17     return 0;
18 }

```

Listing 11: C Program for Fibonacci Sequence using Recursion

**Output Execution Trace for  $n = 5$ :** fibonacci(0)=0, fibonacci(1)=1, fibonacci(2)=1, fibonacci(3)=2, fibonacci(4)=3.  
**Sequence printed:** 0 1 1 2 3

---

## Question 7

Explain the following concepts related to function and pointers in C, with suitable C examples: (i) Function returning a pointer (ii) Passing an array to a function (iii) Pointer to a function (iv) Passing Array of Structure

**Solution / Explanation:**

- (i) **Function Returning a Pointer:** A function whose return type is a pointer (e.g., `int*`). It returns a memory address (typically dynamic memory or static variables).

```

1 int* getMax(int *a, int *b) {
2     return (*a > *b) ? a : b;
3 }

```

Listing 12: Function Returning Pointer Example

- (ii) **Passing an Array to a Function:** Arrays decay into pointers when passed to functions. The base address is passed.

```

1 void printArray(int arr[], int size) {
2     for(int i = 0; i < size; i++) printf("%d ", arr[i]);
3 }

```

Listing 13: Passing Array Example

- (iii) **Pointer to a Function (Function Pointer):** A pointer variable holding the memory entry-point address of an executable function code.

```

1 int add(int a, int b) { return a + b; }
2
3 int main() {
4     int (*fptr)(int, int) = add; // Declare and assign function pointer
5     printf("Sum = %d\n", fptr(5, 3)); // Output: 8
6 }

```

Listing 14: Function Pointer Example

- (iv) **Passing Array of Structure to Function:** An entire collection of structure records passed to a function using pointer or array syntax.

```

1 struct Item { int id; float price; };
2
3 void displayItems(struct Item list[], int count) {
4     for(int i = 0; i < count; i++) {

```

```

5     printf("ID: %d, Price: %.2f\n", list[i].id, list[i].price);
6 }
7 }

```

Listing 15: Passing Array of Structure Example

## Question 8

Compare any two of the following pairs in detail, giving suitable examples: (i) Text file and Binary file (ii) fprintf ()/fscanf() and fwrite()/fread() (iii) Static memory allocation and Dynamic memory allocation

**Solution / Explanation:**

(i) **Text File vs. Binary File:**

| Aspect                     | Text File                            | Binary File                           |
|----------------------------|--------------------------------------|---------------------------------------|
| <b>Data Representation</b> | Human-readable text.                 | Raw binary 0s and 1s stored directly. |
| <b>File Extension</b>      | .txt, .c, .csv                       | .bin, .dat, .exe                      |
| <b>I/O Functions</b>       | fprintf(), fscanf(), fputs()         | fwrite(), fread()                     |
| <b>Storage Efficiency</b>  | Takes more space for numerical data. | Highly compact and fast execution.    |

(ii) **fprintf()/fscanf() vs. fwrite()/fread():**

- **fprintf() / fscanf():** Formatted text file I/O operations converts numbers to text characters during read/write.
- **fwrite() / fread():** Block-level unformatted binary I/O operations transferring raw memory blocks directly to/from disk.

(iii) **Static Memory Allocation vs. Dynamic Memory Allocation:**

| Feature                | Static Allocation              | Dynamic Allocation                    |
|------------------------|--------------------------------|---------------------------------------|
| <b>Allocation Time</b> | At Compile Time.               | At Runtime (Execution Time).          |
| <b>Memory Region</b>   | Stack memory segment.          | Heap memory segment.                  |
| <b>Resizing</b>        | Fixed size; cannot be changed. | Flexible; can resize using realloc(). |
| <b>Management</b>      | Automatic by compiler.         | Manual using malloc(), free().        |

## Question 9

Explain any five string related functions. Write a program for checking palindrome string.

**Solution / Explanation:**

A. **Five Built-in String Functions (<string.h>):**

1. **strlen(s):** Returns the length of string **s** (excluding null character '\0').
2. **strcpy(dest, src):** Copies string **src** into string **dest**.
3. **strcat(dest, src):** Concatenates (appends) string **src** to the end of **dest**.

4. `strcmp(s1, s2)`: Compares two strings lexicographically; returns 0 if equal.

5. `strrev(s)` / Pointer Reversal: Reverses the characters in string `s`.

**B. C Program to Check Palindrome String:** A string is a **palindrome** if it reads the same forward and backward (e.g., "madam", "radar").

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int isPalindrome(char str[]) {
5     int left = 0;
6     int right = strlen(str) - 1;
7
8     while (right > left) {
9         if (str[left] != str[right]) {
10             return 0; // Not a palindrome
11         }
12         left++;
13         right--;
14     }
15     return 1; // Is a palindrome
16 }
17
18 int main() {
19     char str[100];
20     printf("Enter a string: ");
21     scanf("%s", str);
22
23     if (isPalindrome(str)) {
24         printf("\"%s\" is a Palindrome String.\n", str);
25     } else {
26         printf("\"%s\" is NOT a Palindrome String.\n", str);
27     }
28     return 0;
29 }
```

Listing 16: C Program to Check Palindrome String

---

## Question 10

What is recursion? Write a program to find factorial of a number using recursion.

**Solution / Explanation:**

**Recursion** occurs when a function calls itself to solve a smaller instance of the same problem.

**Factorial Mathematical Definition:**  $n! = n \times (n - 1)!$  for  $n > 0$ , with base condition  $0! = 1$ .

```
1 #include <stdio.h>
2
3 // Recursive function to calculate factorial
4 long long factorial(int n) {
5     if (n == 0 || n == 1) {
6         return 1; // Base condition
7     }
8     return n * factorial(n - 1); // Recursive call
9 }
10
11 int main() {
12     int num = 5;
```

```

13     if (num < 0) {
14         printf("Factorial of negative numbers does not exist.\n");
15     } else {
16         printf("Factorial of %d = %lld\n", num, factorial(num));
17     }
18     return 0;
19 }

```

Listing 17: Recursive C Program for Factorial

**Step-by-Step Execution Call Stack Trace for  $n = 5$ :**

| Call Level | Recursive Call Expression                         | Returned Value      |
|------------|---------------------------------------------------|---------------------|
| Level 1    | <code>factorial(5) = 5 * factorial(4)</code>      | $5 \times 24 = 120$ |
| Level 2    | <code>factorial(4) = 4 * factorial(3)</code>      | $4 \times 6 = 24$   |
| Level 3    | <code>factorial(3) = 3 * factorial(2)</code>      | $3 \times 2 = 6$    |
| Level 4    | <code>factorial(2) = 2 * factorial(1)</code>      | $2 \times 1 = 2$    |
| Level 5    | <code>factorial(1) = 1 (Base Case reached)</code> | 1                   |